Source: docs\RAZORPAY_TEST_SETUP.md | Export: 11 November 2026

Razorpay TEST setup and acceptance

Audit date: 8 November 2026. Application commit inspected: `f6448b0`. No gateway transaction, production order, configuration change or deployment was performed by this audit. No payment implementation rewrite is needed for setup.

Current hosted evidence

- Vercel's latest Production deployment is READY for `f6448b0`.
- Public HTTPS `GET /api/payments/webhook` returns 405 with `Allow: POST`.
- An unsigned JSON `POST /api/payments/webhook` returns 503, code `configuration`. The route is reachable but not configured for acceptance.
- Signed-out `GET /api/payments/capabilities` returns 401.
- Vercel's variable-name inventory contains `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET`, both sensitive and scoped to Production. Their values, matching account, TEST prefix and actual gateway validity were not read or proven.
- The remaining Razorpay/worker variables below were absent. An unset LIVE flag fails closed; explicitly save `false` as part of setup.
- The previous trusted local Firebase credential file is unavailable and no replacement is configured locally. Worker metadata and deployed-rule equality could not be independently read. Do not manually fabricate a worker heartbeat.

Exact webhook configuration

Use this URL, without a query string:

`https://www.parinayasarees.com/api/payments/webhook`

In Razorpay Dashboard select **Test Mode**, then **Accounts & Settings → Website and app settings → Webhooks → + Add New Webhook**. Enter the URL, a separate cryptographically random secret, an owner-monitored alert email and these events:

- `payment.authorized`
- `payment.captured`
- `payment.failed`
- `order.paid`
- `refund.created`
- `refund.processed`
- `refund.failed`

The webhook secret is selected/generated by the owner, not the API Key Secret. Generate it privately using a password manager (32 or more random characters, maximum 512). Save the identical value as `RAZORPAY_WEBHOOK_SECRET` in Vercel. No quotes, surrounding whitespace or line breaks. This implementation requires a secret even though the Dashboard describes it as optional. Enable the webhook after the configured deployment is ready. Do not use the webhook URL as a browser checkout callback URL or add a Firebase token to it.

Official references: Dashboard setup, raw-body validation and retries.

Vercel variables

All entries are **server-only**. Never create `VITE_RAZORPAY_*` variables. In Vercel select the existing project → Settings → Environment Variables.

Variable	Value/source	Manual action	Vercel scope
`RAZORPAY_MODE`	Literal `test`	Add	Production
`RAZORPAY_KEY_ID`	Dashboard Test Mode API Key ID, starting `rzp_test_`	Already present; confirm the correct TEST key privately	Production
`RAZORPAY_KEY_SECRET`	Secret paired with that exact TEST API key	Already present; confirm the pair privately	Production, sensitive
`RAZORPAY_WEBHOOK_SECRET`	Same independent random secret saved in this TEST webhook	Add	Production, sensitive

Variable	Value/source	Manual action	Vercel scope
`RAZORPAY_TEST_UIDS`	Comma-separated Firebase Authentication UIDs of explicitly approved test accounts, not email addresses	Add the minimum approved list	Production
`RAZORPAY_LIVE_ACTIVATED`	Literal `false`	Add; keep false	Production; also false wherever testing
`PAYMENT_WORKER_SECRET`	A different cryptographically random secret, at least 32 characters	Add and configure the scheduler with the same value	Production, sensitive
`RAZORPAY_TEST_ENVIRONMENT`	Literal `false` on hosted environments	Optional explicit false; never rely on true for Production	Production optional; local Development may use true

Preview: leave payment credentials unset. Current browser-origin restrictions deliberately exclude `*.vercel.app` previews; merely copying Production variables does not make Preview checkout supported.

Development: only if testing locally, put TEST credentials in ignored `.env.server.local`, with `RAZORPAY_MODE=test`, `RAZORPAY_LIVE_ACTIVATED=false` and either approved `RAZORPAY_TEST_UIDS` or `RAZORPAY_TEST_ENVIRONMENT=true`. Run `npm run dev:api` plus `npm run dev`. Vercel's Development scope is needed only if intentionally pulling local configuration; local files are not created from `.env.example` automatically. Never copy production Firebase credentials into an automated emulator fixture.

`VERCEL_ENV` and `NODE_ENV` are platform/runtime variables, not manually set payment credentials. Currency is fixed to INR with server-calculated integer paise; no Razorpay currency environment variable exists.

Existing Firebase server credentials remain required. Production currently has `FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL`, `FIREBASE_PRIVATE_KEY`. Keep this existing configuration; it explicitly targets `parinaya-silks` / `(default)`. Do not add a second credential mechanism. Existing frontend Firebase settings and Cloudinary configuration also remain unchanged.

After saving server variables, **redeploy Production** so the function receives them. Saving variables alone does not update an existing function instance.

Required recovery worker

Configure a reliable external scheduler with this exact request every minute:

- Method: `POST`
- URL: `https://www.parinayasarees.com/api/payments/worker`
- Header: `Authorization: Bearer <PAYMENT_WORKER_SECRET>`
- Header: `Content-Type: application/json`
- Body: `{}`

Store the authorization value in the scheduler's protected configuration. Do not put it in the URL, logs, job name or source. Allow enough request time for the worker's 35-second work budget (the function limit is 60 seconds). A scheduler service/account must be selected and provisioned; no scheduler is configured by `vercel.json`. An every-minute Cloud Scheduler HTTP job is one option.

The first authorized worker execution writes its own `paymentOperations/worker` heartbeat. Production initiation requires it to be less than five minutes old and bound to the current mode/key ID. A 2xx webhook acknowledgement only proves durable receipt, not capture or subsequent processing. Check worker results and safe diagnostics for deferred events or manual review. Never set a ready flag or heartbeat manually to bypass this gate.

First TEST transaction and acceptance matrix

1. Complete the variables, configured deployment, TEST webhook and scheduler. Privately verify Razorpay automatic capture is enabled; authorization alone is not payment success. Check the actual methods available in the TEST account.
2. Verify the published Firestore rules match the reviewed local protections: browser writes to orders/payment internals are denied; administrator actions still require both `admin:true` and enabled `adminAccess/{uid}`.
3. Choose an explicitly approved Firebase test account and existing, owner-approved test stock/coupon/points data. **Razorpay TEST does not isolate Firestore**: this deployment still reserves inventory and writes orders in the configured production project. Do not test using real sellable stock or customer balances. A separate Firebase test project is not a drop-in environment switch because the server currently pins the project; separate isolation requires a reviewed deployment plan. This audit creates no test customers or inventory.
4. Sign in as an allowlisted test customer on `https://www.parinayasarees.com`. Add the approved test item to Bag, select delivery address, review the current INR total, optionally apply a test coupon/points, and continue to Payment. Online payment must be labelled TEST and available only to approved accounts.
5. Open Razorpay Checkout once. Select Card and use an official current Razorpay test card, for example Visa `4100 2800 0000 1007`, random CVV and a future expiry date. Follow its mock-bank success/OTP flow. Use no real card or banking credentials.

6. Confirm the matching Razorpay TEST order is paid and payment captured for the exact INR amount. Confirm server-verified customer/Admin status, one internal order, one inventory reservation/settlement, correct coupon/points usage and selective Bag cleanup. Do not infer success from the modal callback alone.
7. In Dashboard inspect deliveries of the subscribed webhook events: this endpoint returns 202 only after signature verification and durable event acceptance. Confirm the worker processes the matching event; a sample unrelated provider order can acknowledge safely yet remain unmapped, so use the app-created order.
8. Repeat controlled failure, dismissal, callback/network loss, refresh/resume and duplicate-delivery scenarios. Check payment status before another attempt; unresolved or authorized payments retain their hold. Verify no duplicate charge, order, stock deduction or coupon/points use. Verify COD on separate approved test inventory once the online lock is resolved.
9. Exercise partial/full TEST refunds from Razorpay Dashboard. Confirm pending, failed and processed states are distinct. This app tracks provider refunds but does not initiate refunds, automatically restock, or automatically allocate loyalty refunds. Loyalty-related refunds deliberately hold spending pending an approved accounting allocation workflow; do not clear flags manually.

Use the current official Standard Checkout and test-method documentation for UPI and net-banking scenarios supported by the account/device. No actual Razorpay TEST transaction or real webhook delivery is claimed by local mocked checks.

LIVE activation is a separate approval and must remain disabled throughout.

Checks performed for this audit

All passed on 8 November 2026:

- `npm test`: 185 tests, including configuration, original-body HMAC, request authorization and capture-only confirmation.
- `npm run lint` (TypeScript) and `npm run build` (existing large-chunk warning).
- `npm run test:catalog-runtime` and `npm run test:payments-runtime`.
- Selected existing `test:client-changes` services: `payments-emulator`, `orders-emulator`, `coupons-emulator`, `loyalty-emulator`, `delivery-loyalty-emulator`, `admin-rules`, `customer-rules`, `category-rules`.
- Existing browser regressions: `payments`, `checkout-design`, `checkout-coupons`, `checkout-context`. Payment browser requests are mocked; no real Razorpay endpoint is called. Responsive payment layouts cover 320/390/768/1440px, with additional checkout widths.
- `git diff --check`; frontend output has no Razorpay server-secret variable names or private-key markers. No secret values were printed or added.

The emulator payment matrix verifies one provider creation, signature/capture separation, failures and dismissal recovery, durable event retries, stock/coupon/ points consistency, customer/Admin projections, partial/full refund evidence, ownership and the Production worker/allowlist gate. These checks do not verify the newly added Vercel credential values, a live scheduler, actual TEST payment methods, a signed Razorpay delivery or production rule equality.

Only this guide, a link in `docs/PAYMENTS.md` and comments in `.env.example` were prepared. No application code, payment flag, security rule, production record, commit, push or deployment was changed.